L/S Computers

Split cache < data / instr

Example

- 20% L/S instr.
- IMR = 8% (instr)
- DMR = 10% (data miss rate)
- Miss Penalty = 25 ns
- Block rate = 3 GB/s
- $CPI_{ideal} = 300$
- $t_{ac} = 1\,CC$
- IC = 1000

a) AMAT = ?

$$CCT = \frac{1}{CR} = \frac{1}{3 \cdot 10^9} = \frac{10^{-9}}{3} = 0{,}333\ ns$$

$$\text{Miss Penalty}^{CC} = \left\lceil \frac{25\ ns}{0{,}333\ ns} \right\rceil = \lceil 75{,}075 \rceil = 76$$

$$\Rightarrow AMAT = t_{ac} + MR \cdot \overrightarrow{MP}^{t} = 0{,}33 + 0{,}083 \cdot 76 \cdot 0{,}33 = 2{,}434\ ns$$

$$\text{Misses/instr} = 1 \cdot \underbrace{0{,}08}_{\text{IMR - toate}} + \underbrace{0{,}2 \cdot 0{,}1}_{\text{DMR} \cdot \text{doar L/S}} = 0{,}1$$

$$MR = \frac{\text{Misses/instr}}{\text{Mem. accesses/instr}} = \frac{0{,}1}{1 + 0{,}2} = 0{,}083$$

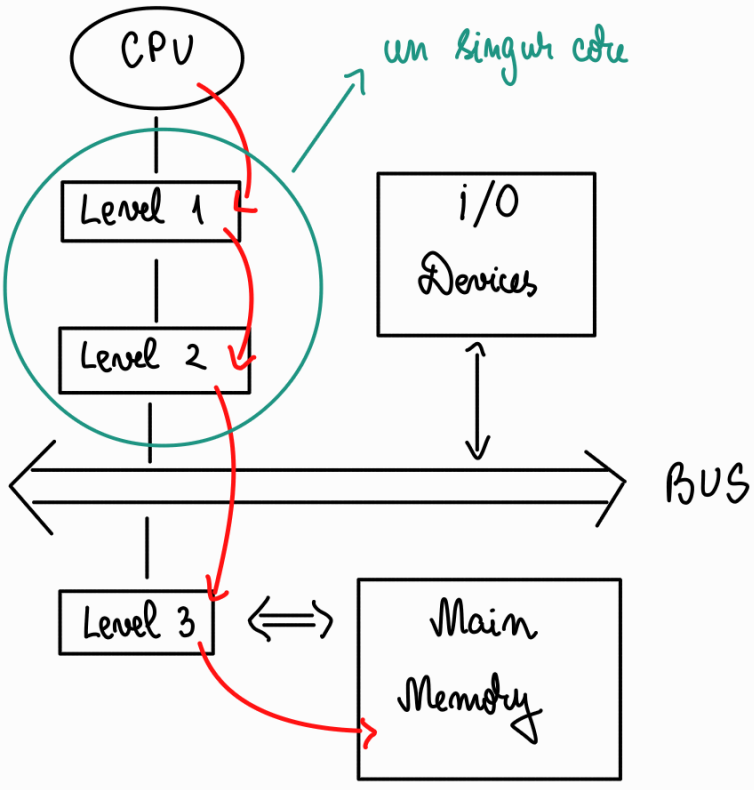
(1 ↑ toate; 0,2 ↑ L/S → au nevoie de acces in plus)

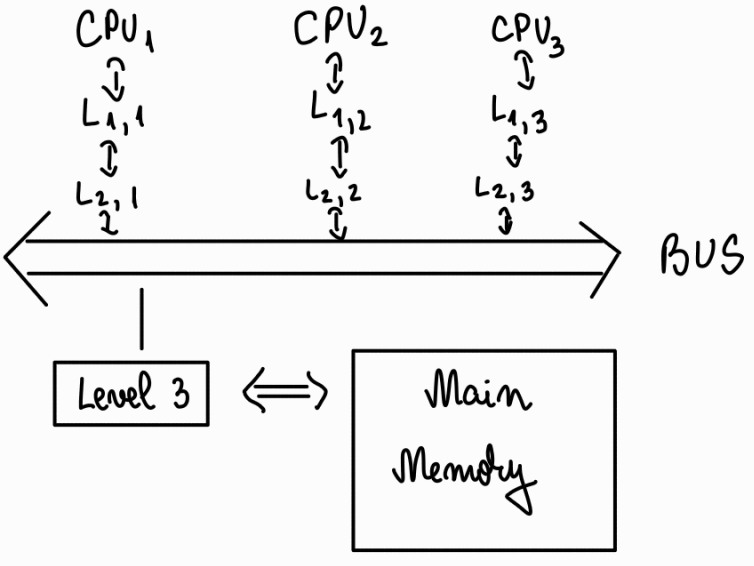
b) $CPU_{time} = ?$

$$CPU_{time\ real} = IC\left(CPI_{ideal} + MR \cdot \overrightarrow{MP}^{CC} \cdot \text{Mem. acc./instr}\right) \cdot CCT =$$

$$= 1000(3 + 0{,}1 \cdot 76) \cdot 0{,}33 = 3529{,}8\ ns$$

3.5. Multi - level caches





(1.) Sist. de memorie cu 2 nivele cache → CPI ideal = 1 CC

CR = 4 GHz → CCT = $\frac{1}{4 \cdot 10^9}$ = 0,25 ns

MP = 100ns

Miss Rate / instr = 2%

t_{ac} = 5ns

Miss Rate / $instr_2$ = 0,5% după ce adăugăm L_2

De câte ori va fi mai rapid procesorul și cu L_2

- doar cu L_1

$$CPU_{time\ 1\text{-}level} = IC\,(CPI_{ideal} + \underbrace{Mem\ accesses/instr \cdot MR \cdot MP}_{MR/instr}) \cdot CCT =$$

$$= IC \cdot \left(1 + 0{,}2 \cdot \left\lceil \frac{100\ ns}{0{,}25\ ns} \right\rceil\right) \cdot 0{,}25\ ns =$$

$$\left\lceil \frac{100\ ns}{0{,}25\ ns} \right\rceil \downarrow 400\ cc$$

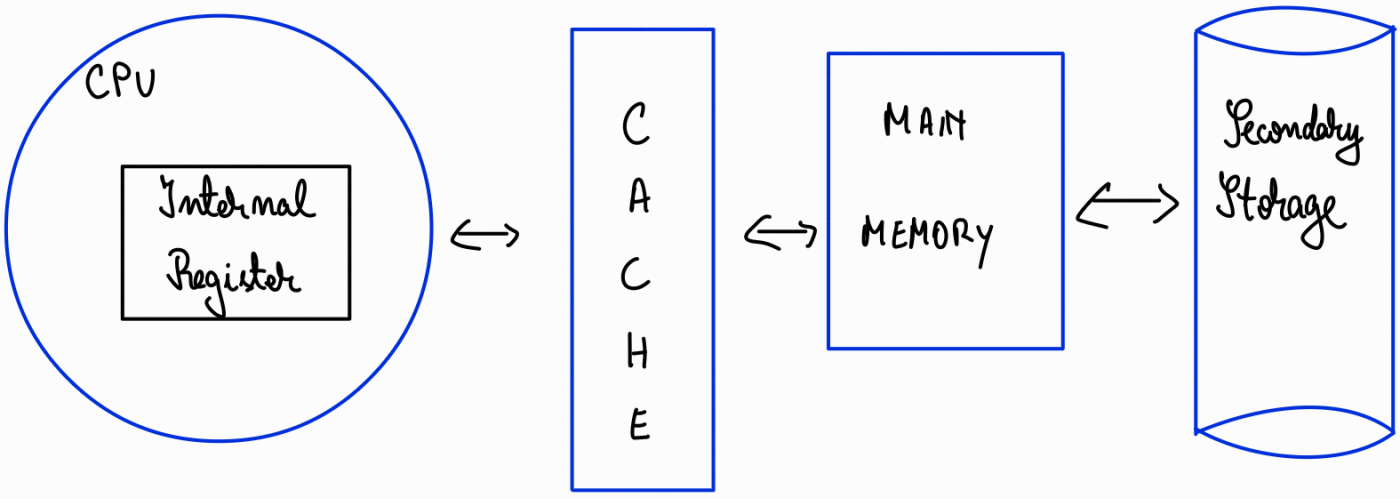
$$= IC \cdot 2{,}25\ ns$$

$$CPU_{time\ 2\text{-}level} = IC\left(1 + \underbrace{0{,}005 \cdot 400}_{L_2} + 0{,}02 \cdot \left\lceil \frac{5\ (tac)}{0{,}25\ ns} \right\rceil_{L_1 \to L_2 \,=\, 20\ cc}\right) \cdot 0{,}25 =$$

$$= IC \cdot 0{,}85$$

$$\frac{Perf\ L_2}{Perf\ L_1} = \frac{2{,}25}{0{,}85} = 2{,}65 > 1 \Rightarrow Perf\ L_2 \text{ e mai bună}$$

4. Virtual Memory



4.1. Motivation

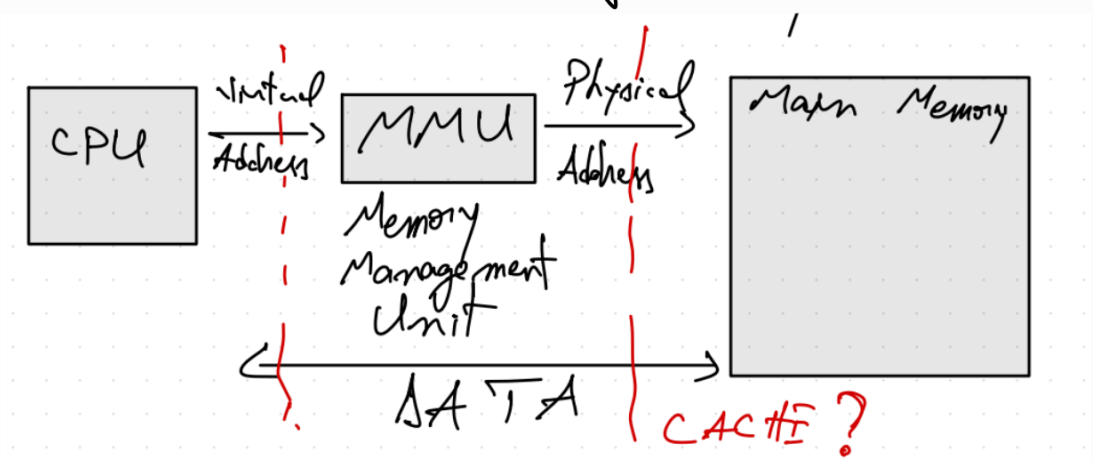
Virtual Machines, probleme puse pt. implementarea lor:
1. Protect processes from one another
2. Share the limited memory among processes (VMs)

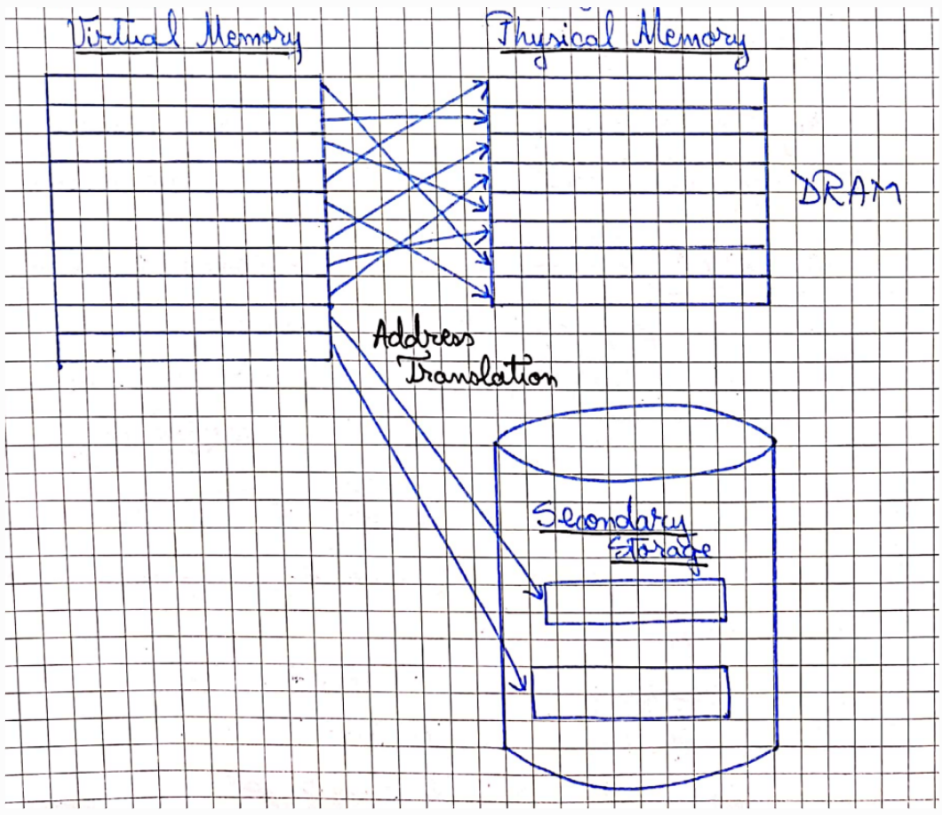
Pentru asta, programatorii foloseau overlay, dar acum se foloseste o masina virtuala ce foloseste un spatiu de memorie virtuala (SMV) ce urmeaza a fi mapat pe adresele fizice.

SMV se fragmenteaza in: Segments (variable size)
Pages (fixed size) → M.M.
↑ cel care ne intereseaza pe noi

virtual memory space ≥ physical mem. space

Virtual address —translation→ Physical addres





Huge Miss Penalty → page fault

very low miss rate → full associative

→ table → page table

→ fiecare proces are pagina lui

→ WT este exclus

→ WB preferat

→ replacement → software

ex. address translation pe ARM v8

1 word = 64 bits = 8B = 2^3B

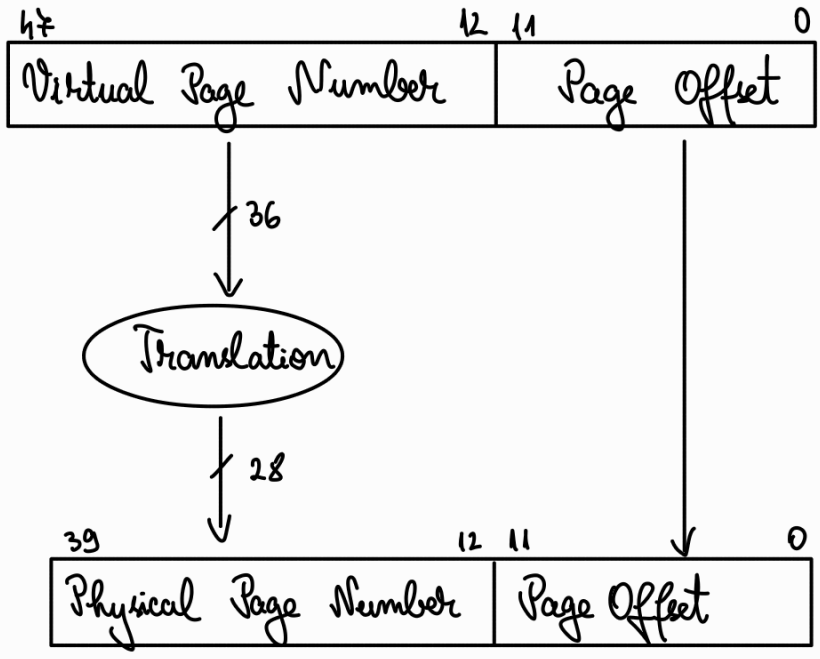
16 MSB are unused

64 - 16 = 48 de biți

page size = 4 KiB = $2^2 \cdot 2^{10}$ B = 2^{12} B ⇒ page offset = 12 bits

physical memory size = 1 TiB = 2^{40} B = 2^{28} pages ⇒ physical address = 40 biți
(P.M.) după translation 28 biți

virtual memory size = 256 TiB = $2^8 \cdot 2^{40} = 2^{48}$ B = 2^{36} pages ⇒ virtual address = 48 biți
înainte translation = 36 biți



Paginile virtuale se mapează în memoria fizică doar când procesul este activ (posedă CPU la un moment dat)

Denumiri: page → granule ← Memory Management Unit
page fault → MMU exception
↳ când pagina nu e în MM și trebuie adusă de pe disc prin mecanismul de memory swapping

Problemă luată în considerare pt. construirea unui MMU: Misses cost a lot! (Secondary storage is 100 000 x slower than MM). Rezolvări:

① relatively big pages: 4KiB ÷ 64KiB
Servers → 32KiB ÷ 64KiB
Embedded → 1KiB ÷ 4KiB

② WT e imposibil → Folosim doar WB

③ Replacement is decided in software (nu mai folosesc metode random) — approximate LRU

④ Nu îți poți permite să cauți tag-uri →
=> Folosim doar full-associativity mapping cu Page Tables